Kick Start 2019 - Round F

Analysis: Flattening

Test set 1 (Visible)

We can start with an interesting observation, whenever we rebuild a section of the wall, it is equivalent to removing that section. So we can follow two steps,

1. Pick a set of sections of the wall to remove.
2. Check if in the remaining wall sections, the number of positions where $\mathbf{A}_i \neq \mathbf{A}_{i+1}$ are less than or equals **K**.

There are $2^{\mathbf{N}}$ possible sets of sections, and for each set, checking if the remaining sections will make Blotch happy or not will take O(**N**) time. So the time complexity of this approach is O($2^{\mathbf{N}} \times \mathbf{N}$), which is fast enough for test set 1.

Test set 2 (Hidden)

We need to approach the problem a bit differently for test set 2. We can start with observing the amount of walls we need to remove for a particular range of consecutive wall sections[i...j]. The most optimal way of making a set of consecutive wall sections have same height would be to figure out which height appeared the most, and remove all sections with a different height.

So for each consecutive set of walls, we can calculate number of wall removals needed. Let's say, R(i, j) defines number of removals necessary to have all wall sections from i to j have the same height.

We can define a solution based on a function F(x, k), where F(x, k) denotes the number of the walls we need to remove so that the sections from 1 to x in the input has $\mathbf{A}_i \neq \mathbf{A}_{i+1}$ in at most k places. We can define the recurrence relation as, F(x, k) = min(F(i, k-1) + R(i+1, x)) for all $1 \leq i \leq x-1$. The minimum number of walls that we need to remove for given **N** and **K** is F(**N**, **K**). We can compute this function using dynamic programming, which will have O($\mathbf{N}^2 \times \mathbf{K}$) time complexity, which is fast enough for test 2.

We can have a faster solution, if we decompose the problem from a slightly different angle. We can think about running binary search on number of removals needed to satisfy the condition. In each iteration of binary search, we need to calculate if it is possible to have k or less number of positions where $\mathbf{A_i} \neq \mathbf{A_{i+1}}$ if we have at most x removal, x being the value of binary search value of that iteration. That can be also calculated with another dynamic programming, leading us to a solution with time complexity of O($\mathbf{N}^2 \times \log(\mathbf{N})$).

Kick Start 2019 - Round F

Analysis: Spectating Villages

Test set 1 (Visible)

Since there are very few villages for this set, we can calculate the sum of beauty values of illuminated villages for every possible set of villages we choose to build lighthouses in and then take the maximum among these.

For each such set, we have to find which villages will be illuminated. So, for each village, we see if it is in the set or if any of its neighbours are in the set and add its beauty value accordingly.

Since there will be $2^{\mathbf{V}}$ such sets and for each such set we are taking O(**V**) time to figure out what is the corresponding total beauty value for the set, the overall complexity of this approach is O($\mathbf{V}2^{\mathbf{V}}$)

Test set 2 (Hidden)

The the graph formed by taking the villages as nodes and roads as edges is a tree. Let's root this tree at node number 1.

Now, let us define a function maxBeauty(K, P, Q) which represents the maximum beauty value we can obtain from nodes in the subtree of node K (including itself) such that P is a boolean indicating whether the parent node of K has a lighthouse and Q is a boolean indicating whether K itself has a lighthouse. The solution to our problem is simply max(maxBeauty(1, 0, 1), maxBeauty(1, 0, 0)). We consider a few cases to evaluate maxBeauty(K, P, Q).

If Q = 1, then we have a lighthouse placed at K. Which means all of its children will be illuminated irrespective of them having a lighthouse or not. Therefore, in this case, maxBeauty(K, P, Q) = $\mathbf{B}_K$ + sum of max(maxBeauty(C, 1, 0), maxBeauty(C, 1, 1)) for all children C of node K.

If Q = 0 and P = 1, then irrespective of whatever we choose for the children of K, they are not going to recieve light from K but K itself is going to be illuminated. Therefore, in this case, maxBeauty(K, P, Q) = $\mathbf{B}_K$ + sum of max(maxBeauty(C, 0, 0), maxBeauty(C, 0, 1)) for all children C of node K.

Else, we have Q = 0 and P = 0. This means that the children of K are not going to recieve light from K but the illumination of K depends on whether we place a lighthouse in at least one of the children of K.
This case can be handled using a dynamic programming approach. We define dp[i][j] as maximum sum of beauty values of all illuminated nodes in the first i subtrees of node K such that, if j = 0 then we have not placed a lighthouse in any of the first i children of K and if j = 1 then there is at least one node among the first i children of K with a lighthouse.
Therefore, dp[i][0] = dp[i-1][0] + maxBeauty(C_i, 0, 0) and
dp[i][1] = max(dp[i - 1][1] + max(maxBeauty(C_i, 0, 0), maxBeauty(C_i, 0, 1)), dp[i - 1][0] + maxBeauty(C_i, 0, 1)).
(Here, C_i represents the ith child of K)
Finally, for this case, maxBeauty(K, P, Q) = max(dp[M][1] + $\mathbf{B}_K$, dp[M][0]) where M is the total number of children of K.

Since for every node K we can get maxBeauty(K, P, Q) for all values of P and Q in O(M) time, overall complexity of this approach is O(**N**).

Kick Start 2019 - Round F

Analysis: Teach Me

Test set 1

We can solve this test set by considering all ordered pairs of employees. For an ordered pair (i, j), we can check whether there is a skill that the i-th employee knows that the j-th employee does not know with a simple O($\mathbf{C_i} \times \mathbf{C_j}$) check using nested loops.

This solution runs in O($5^2 \times \mathbf{N}^2$).

Test set 2

We can solve this test set by defining m(i) as the number of employees who can mentor the i-th employee. If we can compute m(i), the answer to the problem is the sum of all m(i).

To compute m(i), we can count the number of employees who **cannot** mentor the i-th employee instead. We can observe that the j-th employee cannot mentor the i-th employee if and only if the set of skills known by the j-th employee is a subset of the set of skills known by the i-th employee. Therefore, we would like to count the number of employees whose set of skills is a subset of the set of skills known by the i-th employee.

To count this, we can consider every subset of {$\mathbf{A_{i1}}$, ..., $\mathbf{A_{iC_i}}$}. For a subset B, we can count the number of employees whose set of skills is exactly B. Taking the sum of all subsets gives us the number of employees whose set of skills knowledge is the subset of the set of skills known by the i-th employee. m(i) is simply the number of employees subtracted by this value.

To be able to compute the number of employees whose set of skills is exactly a given set, we can preprocess the set of skills into an occurences hashmap. In other words, we can keep a hashmap that takes a set of skills as its key and returns the number of employees who knows the exact same set of skills as its value.

For each employee i, we need to consider every subset of {$\mathbf{A_{i1}}$, ..., $\mathbf{A_{iC_i}}$}. Since there can be up to 2^5 subsets, this solutions runs in O($2^5 \times \mathbf{N}$).